

ChatDBG Pro: Disentangling Model Scale from Tool Surface in LLM-Driven Debugging

Anonymous ACL submission

Abstract

Large language models are increasingly being used as debugging assistants, but it remains unclear how much debugging capability is attributable to model capabilities versus information access (tool calling and context retrieval). We extend ChatDBG, a debugger interface that pairs LLM capabilities with GDB tool calls, by evaluating whether smaller, cost-efficient models can match frontier performance when given appropriate tool access. We evaluate four models—GPT-4, Llama-3.1-8B, Nemotron 30B, and Qwen 30B—across synthetic and original paper-derived crash-debugging scenarios scored on root cause identification and fix accuracy. We find that model scale is not the primary determinant of debugging success, and that iterative investigation can allow smaller models to debug at the level of or even beat frontier models. We further evaluate all four models on 156 real-world C/C++ bugs from BugsCPP under a GDB-only tool configuration and find that every model, including GPT-4, scores near 0% when only given GDB access, while configurations expanding tool access significantly improve debugging capability.

1 Introduction

Debugging consumes a significant fraction of programmer time, and recent work has tried to reduce that cost by integrating large language models into the debugger itself. The clearest example is **ChatDBG** (Levin et al., 2025), which embeds a chat interface inside Pdb, LLDB, and GDB so that a developer can ask plain-English questions while a program is paused at a crash. The system has been downloaded more than 75,000 times and is, to our knowledge, the only published debugger plugin in which the model can autonomously issue debugger commands. On 22 unpublished Python programs, ChatDBG produced an actionable diagnosis 67% of the time on a single targeted query

and 85% with one follow-up; on 8 C/C++ programs from BugBench and BugsC++, it identified the root cause in 36% of cases and the proximate cause in another 55%. Every reported number used gpt-4-1106-preview, at roughly \$0.12 per Python session and \$0.06 per C/C++ session.

These results establish that the interface works, but they leave the underlying science underspecified. Four limitations narrow the conclusions one can draw. First, the benchmark is small (30 bugs total), single-file, and dominated by memory errors, so it cannot distinguish bugs that are easy from bugs that are hard, and it provides no signal about multi-file projects of the kind covered by SWE-bench (Jimenez et al., 2024). Second, only one model is evaluated; there is no smaller-model or open-weights baseline, even though much of a debugger’s work (printing variables, walking frames, following pointers) is mechanical and may not require a frontier model. Third, the evaluation is human-in-the-loop: a person types the prompt, refines it when the first answer is weak, and judges whether a fix is “actionable,” which makes the success rates difficult to compare against autonomous agents. Fourth, the system explains but never executes; the model neither verifies its proposed fix nor falls back on a code interpreter when computation would be cheaper than reasoning.

A separate observation sharpens the question. Mini-swe-agent (Yang et al., 2024) resolves over 74% of SWE-bench Verified using only a bash shell with no structured tool API. If a generic shell suffices for repository-level bug fixing, it is not obvious that a structured GDB surface contributes anything on top. Combined with the limitations above, this suggests that the interface, the model, the debugger surface, and the human in the loop are entangled in the original evaluation, and that disentangling them requires a larger and more controlled benchmark.

Contributions and research questions. We re-

build the ChatDBG harness, scale its evaluation corpus to over 100 multi-file C/C++ bugs drawn from BugsC++ and YARA, and run four ablations:

- **RQ1: Tool surface.** Does a GDB-only harness produce different outcomes than a bash-only harness on identical bugs?
- **RQ2: Model scale.** Can open-weights models in the 30B–120B range (Mamba-3, Nemotron Nano, Qwen 30B, Nemotron Super) match GPT-4 on this task, and on which steps does the gap appear?
- **RQ3: Difficulty.** How does tool-use behavior change as bugs move from single-file memory errors to multi-file logical bugs?
- **RQ4: Interface extensions.** What is the marginal effect of richer stack-trace context, chain-of-prompt self-querying, frontier-as-a-tool delegation, in-loop code execution, and a streaming Mamba policy that maintains hidden state across turns?

The remainder of the paper describes our reproduction harness (§2), the benchmark corpus (§2.1), the four ablation studies, and what those studies imply about where compute should be spent in an LLM-driven debugger.

2 Methods

The original ChatDBG result of 67% actionable-fix rate (Levin et al., 2025) fixes a single configuration: GPT-4 driving a sanitized gdb interface across nine single-file C/C++ crashes. Two of the three relevant variables, the model and the tool surface, are not independently controlled, so the result does not separate model capability from interface design. We address this with a factorial sweep over three axes: the buggy program (~ 30 cases), the tool surface (four tiers), and the model (five under test), evaluated end-to-end by an automated three-axis rubric. The sweep yields $\sim 30 \times 4 \times 5 \approx 600$ graded runs, plus targeted ablations on context length, gdb sandbox policy, and prompt enrichment.

2.1 Test Corpus

Cases must crash deterministically, expose the bug to runtime inspection, and have a known ground-truth fix; correctness or performance bugs that do not crash are out of scope. All cases are C or C++,

both for direct comparability with ChatDBG’s original evaluation and because manual memory management is the regime that exercises an interactive debugger.

The corpus contains ~ 30 cases in the headline matrix and ~ 50 in the extended sweep, drawn from six sources: nine direct ports of ChatDBG’s original synthetic suite (bench/cases/paper/); ten additional hand-authored synthetic crashes covering bug classes underrepresented in the original (signed/unsigned loops, integer-overflow allocation, double-free on error paths, vector-iterator invalidation); four BugBench programs (Lu et al., 2005); eleven external-benchmark cases comprising six from CrashBench (Ortega, 2024) and five from the Juliet test suite (National Institute of Standards and Technology, 2017) covering CWE-121, 122, 126, 415, and 416; five injected-repo cases derived by reverting known patches in pinned versions of cJSON, Lua, Mongoose, the SQLite shell, and zlib, in the spirit of the repository-level reversion methodology of SWE-bench (Jimenez et al., 2024); and five BugsCPP libtiff cases (An et al., 2023).

This expansion is motivated by two considerations. First, the original nine-case suite has been on GitHub since 2023 and is plausibly in the pre-training mix of every model evaluated; one memorized fix shifts a nine-case score by 11 points. Second, with five models and four tiers, a nine-case corpus produces noise wider than the effects of interest. Hand-authored cases were written in 2026 and remain unpublished, postdating every model’s training cutoff. The injected-repo cases reuse bugs whose public patches predate cutoffs, but their failing input, crash signature, and exact repository state are novel; pattern recognition is plausible while verbatim retrieval is not.

Each case is described by a case.yaml specifying language, source, build flags, run arguments, expected-crash flag, ground-truth bug metadata, and a criteria block carrying three pre-registered scoring rubrics (§2.5). The criteria are visible to the judge but not to the model under test.

2.2 Tool-Surface Tiers

A tier fixes a tool surface; the model, prompt scaffolding, context window, and sandbox image are held constant across tiers, so any tier-vs-tier difference is attributable to the interactive surface alone.

Tier 1 (bash only). A mini-swe-agent harness (Yang et al., 2025) exposing a single bash tool. The model has no debugger and must reproduce or instrument via the shell. Tier 1 is the no-debugger baseline.

Tier 2 (bash + persistent gdb). The same mini-swe-agent harness with an added gdb tool dispatched to a long-lived gdb subprocess (LocalGdbBashEnvironment). State persists across calls. Tier 2 minus Tier 1 isolates the marginal value of an interactive debugger over a generic shell.

Tier 3 (ChatDBG native). ChatDBG’s original surface: a sanitized debug tool, get_code_surrounding, and find_definition; no shell. This is the closest replication of the paper. As an ablation we run *Tier 3-unfenced*, in which CHATDBG_UNSAFE=1 lifts the gdb command allow-list (break, run, continue, disassemble, x, set, call, full backtrace), measuring the cost of ChatDBG’s safety sanitizer.

Tier 4 (Claude Code CLI). The Claude Code CLI run with `-bare`, providing bash, file read, and file edit. Tier 4 has strictly more tools than the others and is reported as a frontier-agent ceiling, not a controlled comparison.

All tiers run inside Docker containers (synthetic-runner for synthetic cases, gdb-<project> for BugsCPP and injected-repo cases). Patch git history is stripped at workspace prep to prevent agents from recovering the developer fix via `git log`, a leak path identified during pilot runs. Tier configurations are declarative JSON files under `bench/configs/`.

2.3 Models

To test whether frontier scale is necessary for tool-mediated debugging, the model lineup spans the cost-and-scale spectrum rather than concentrating at the frontier. We evaluate two frontier closed models, one frontier-small closed model, and two open-weights $\sim 30\text{B}$ mixture-of-experts models (Table 1). All models are accessed through LiteLLM at temperature 0 (or the provider minimum), with a tier-calibrated max-tokens cap, so prompt and response handling is uniform across providers. GPT-4o is held back as the judge (§2.6) and never appears under test.

Model (provider)	Role	Active params	Cu
gpt-5.5 (OpenAI)	frontier	undisclosed	20
claude-sonnet-4.5 (Anthropic)	frontier	undisclosed	20
gemini-3.1-flash-lite (Google)	frontier-small	undisclosed	20
nemotron-3-nano-30b-a3b (NVIDIA)	open MoE	3B / 30B	20
qwen3-30b-a3b-instruct (Qwen)	open MoE	3B / 30B	20

Table 1: Models under evaluation. All cutoffs predate or coincide with the hand-authored cases in §2.1.

2.4 Run Procedure

A run is identified by the tuple (*case*, *tier*, *model*, *context_lines*, *seed*), hashed into a deterministic *run_id* so re-execution is a cache hit and partially completed sweeps resume safely. Each run compiles the case in a clean Docker workspace (with bug patch applied and git history stripped, for injected-repo cases), runs the binary to crash to capture sanitizer output and backtrace, then enters the agent loop until a final response is emitted or a step, cost, or wall-clock budget is exceeded.

The system prompt (instructions/default.txt) provides generic debugging instructions and descriptions of the active tier’s tools; the bug category, ground-truth lines, and scoring criteria are withheld. The user prompt (build_initial_prompt) supplies six fields: case metadata, an enriched stack trace (top three frames with $\pm\text{context_lines}$ of source, default 10), the sanitizer or crash message, the invocation command line and stdin, the debugger command history, and a fixed question, “*What is the root cause of this crash? Walk through the program state, identify the defect, and propose a fix in code.*” The enriched stack trace is delivered identically on every tier, so tier-vs-tier differences isolate the interactive surface and the *context_lines* ablation isolates static context.

2.5 Scoring Rubric

The original paper grades a single binary axis: did the model identify the root cause? This collapses three distinct outcomes (understanding, symptom-level patching, and structural fix) and obscures the failure mode most relevant to the small-model debugging hypothesis, in which a model localizes correctly but proposes the wrong kind of fix. We score every response on three independent 0/1 axes:

1. **Root cause:** the response correctly diagnoses the defect (right line, variable, and invariant).

2. **Local fix:** the proposed change resolves the proximate symptom at the crash site.
3. **Global fix:** the proposed change addresses the bug class structurally (API, type, or invariant), not just the crashing line.

The axes are independent; local without global is the most common outcome, and global without local is rare. The criteria block is pre-registered per case and visible only to the judge. One limitation: the rubric scores the model’s final response as text, without compiling and re-running an extracted patch. Strict patch execution would penalize prose explanations that are correct but not in a directly-applicable format; the cost is that scores reward articulated correctness rather than verified correctness.

2.6 LLM Judge

Manual grading does not scale to ~600 responses, and pattern-matching is too brittle (line numbers shift; correct rationales use synonyms). We use an LLM judge, following the methodology of Zheng et al. (2023), which characterizes the three biases relevant here: position, verbosity, and self-preference.

The judge (bench/judge.py) loads each run’s case.yaml, result.json, and buggy source, then issues a strict-mode prompt (prompts/judge_system.txt) requiring 0/1 scores with no partial credit and a two-sentence rationale per axis. The source is truncated at 20,000 characters and the response at 40,000. The judge returns a single JSON object {root_cause, local_fix, global_fix, rationale} written to a sibling score.json with judge token counts for cost auditing.

We use GPT-4o as the judge for four reasons. Its October 2023 training cutoff predates every model under test, removing recency-driven self-preference. It is not in the model-under-test set, removing direct self-preference. It follows JSON-schema constraints reliably (parse-failure rates near zero in pilot). Its biases are documented in the calibration literature (Zheng et al., 2023; Gu et al., 2024), so they can be partially corrected. Bias mitigations follow from the taxonomy: the judge sees only the model’s final response, never the debugger transcript, removing transcript-length bias; the strict 0/1 protocol with bounded rationales discourages hedging; and we hand-grade ~10%

of responses with a target judge–human Cohen’s $\kappa \geq 0.7$ before relying on the headline numbers.

Limitations

Ethics Statement

Acknowledgements

References

- Gabin An, Minhyuk Kwon, Kyunghwa Choi, Jooyong Yi, and Shin Yoo. 2023. [BugsC++: A highly usable real-world defect benchmark for C/C++](#). In *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*.
- Jiawei Gu, Xuhui Jiang, Zhichao Shi, Hexiang Tan, Xuehao Zhai, Chengjin Xu, Wei Li, Yinghan Shen, Shengjie Ma, Honghao Liu, Yuanzhuo Wang, and Jian Guo. 2024. [A survey on LLM-as-a-Judge](#). *arXiv preprint arXiv:2411.15594*.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. [SWE-bench: Can language models resolve real-world GitHub issues?](#) In *Proceedings of the 12th International Conference on Learning Representations*.
- Kyla Levin, Nicolas van Kempen, Emery D. Berger, and Stephen N. Freund. 2025. [ChatDBG: Augmenting debugging with large language models](#). *Proceedings of the ACM on Software Engineering*.
- Shan Lu, Zhenmin Li, Feng Qin, Lin Tan, Pin Zhou, and Yuanyuan Zhou. 2005. [BugBench: Benchmarks for evaluating bug detection tools](#). In *Workshop on the Evaluation of Software Defect Detection Tools*.
- National Institute of Standards and Technology. 2017. [Juliet test suite for C/C++, version 1.3](#). NIST Software Assurance Reference Dataset.
- Alfredo Ortega. 2024. [Crashbench: An LLM benchmark to measure bug-finding and reporting capabilities](#). <https://github.com/ortegaalfredo/crashbench>.
- John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. [SWE-agent: Agent-computer interfaces enable automated software engineering](#). In *Advances in Neural Information Processing Systems 37*.
- John Yang, Kilian Lieret, Carlos E. Jimenez, Alex Wettig, Klaus Greff, Aleksandar Botev, Ofir Press, and Karthik Narasimhan. 2025. [mini-swe-agent: The 100 line AI agent that solves GitHub issues](#). <https://github.com/SWE-agent/mini-swe-agent>.
- Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. [Judging](#)

LLM-as-a-Judge with MT-Bench and Chatbot Arena.	365
In <i>Advances in Neural Information Processing Systems 36 (NeurIPS)</i> .	366
	367